

2

▶ Run the sketch.

```
function setup() {
  createCanvas(400, 400);
  background(205);
}

function draw() {
  line(width/2,
        height/2,
        random(0, width),
        random(0, height));
}
```

Start a new sketch at editor.p5js.org.

Move `background(205)` into `setup()`.

Inside the `draw()` function, draw a line from the center of the canvas to a random point on the screen.

1

Start with random steps

P5.JS ACTIVITY GUIDE

First steps with a
RANDOM WALK

by Aleksandra Jovanić

Inspired by Manfred Mohr's iconic 1969 works P-10 and P-18, both *random walks*.

3

Now each new step starts where the previous one ended.

```
function draw() {
  px = x;
  py = y;
  x = random(0, width);
  y = random(0, height);
}
```

Then, inside `draw()`, after drawing the line, store the current position as the previous position and calculate a new random position for the next step.

```
x = px = width/2;
y = py = height/2;
```

Initialize them inside `setup()`, starting at the center of the canvas:

```
let x, y, px, py;
```

2

Connect the steps

7

The final sketch

```
let x, y, px, py;
function setup() {
  createCanvas(400, 400);
  background(245);
  x = px = width/2;
  y = py = height/2;
}

function draw() {
  let s = random(20, 50);
  line(px, py, x, y);
  px = x;
  py = y;
  x = x + (floor(random(0,3))-1)*s;
  let dirY = (floor(random(0,3))-1);
  if (dirY == 0) {
    strokeWeight(4);
  } else {
    strokeWeight(1);
  }
  y = y + dirY*s;
  x = constrain(x, 0, width);
  y = constrain(y, 0, height);
}
```

EXPLORE MORE

Define new set of rules for the next steps. Try moving in any direction by choosing a random angle and using `cos()` and `sin()`:

```
let angle = random(0, TAU);
x += cos(angle)*step;
y += sin(angle)*step;
```

4

Now the step can move in either direction: sometimes up, sometimes down.

```
y = y + (floor(random(0,2))*2-1)*30;
```

The `floor()` function removes the decimal part of a number, giving us an integer.

Or, we can use a multiplier that alternates between -1 and 1.

We can add more `if...else` branches to allow movement in the opposite directions.

```
function draw() {
  px = x;
  py = y;
  if (random(0,1)<0.5) {
    x = x + 30;
  } else {
    x = x - 30;
  }
  y = y + 30;
}
```

3

Right angle turns

6

Vary the path style

The rules don't have to control only movement.

At each step, we can also change the visual properties of the line, for example, its stroke weight, color, or any other parameter we choose.

```
strokeWeight(5);
stroke(random(0,255));
```

By connecting visual changes to the rules of the random walk, we can create results like the one shown here:

5

Move diagonally

What if we also want the walk to move at 45°?

Instead of alternating between -1 and 1, we can also allow 0 as a possible multiplier. At each step, we calculate new values for both `x` and `y`.

This means the walk can move diagonally, left, right, up, or down.

Occasionally, both values will be 0, so there will be no movement at all, but we can live with that.

Instead of an `if...else` branch, we can calculate the new `x` and `y` values directly:

```
x = x + (floor(random(0,3))-1)*step;
y = y + (floor(random(0,3))-1)*step;
```

4

Stay inside the canvas

5

EXPLORE MORE

Go to the [P5.js Reference](#) and look up the `constrain()` function. Try different minimum and maximum values as arguments.

```
function draw() {
  let step = random(20, 50);
  line(px, py, x, y);
  px = x;
  py = y;
  if (random(0,1)<0.5) {
    x = x + (floor(random(0,2))*2-1)*step;
  } else {
    x = x - (floor(random(0,2))*2-1)*step;
  }
  y = y + (floor(random(0,2))*2-1)*step;
  y = constrain(y, 0, height);
  x = constrain(x, 0, width);
}
```